Systems Programming in C

Complete study notes

Covers every slide in **SPC1 - Introduction** and **SPC2 - Expressions and Basic Data Types**, with worked examples, tables, and a practice set with answers.

KEY IDEA

How to read these notes. Content from your slides is the backbone. Green **BEYOND THE SLIDES** boxes add clarifying facts that are not on the slides (handy for exams and debugging), and are marked so you can tell the difference. Red **WATCH OUT** boxes flag common traps.

Contents

1. C in Context 5

1.1 Lecture overview . . . 5

1.2 Very brief history . . . 5

1.3 Design principles . . . 6

1.4 Paradigm: purely procedural . . . 6

1.5 Resources and workflow . . . 7

2. Hello, World! Dissected 8

2.1 Comments . . . 8

2.2 The `main` function . . . 9

2.3 `printf` . . . 9

2.4 Including header files . . . 9

2.5 Compile and execute . . . 9

3. A Second Program: Sum from 1 to n 10

3.1 Statements, functions and variables (the summary slide) . . . 11

3.2 Self-study slides . . . 11

4. Expressions in C 14

5. Basic Data Types, Constants and Variables 14

5.1 A few basic data types . . . 14

5.2 Constants (of basic types) . . . 14

5.3 Variables . . . 15

6. Operators 15

6.1 The operator families . . . 15

6.2 Precedence and associativity . . . 16

6.3 Assignment operators . . . 16

7. Integer Data Types 17

7.1 Sizes and ranges: x86_64 / arm64 (64-bit, "what we all have") . . . 17

7.2 Sizes and ranges: i386 (32-bit) . . . 18

7.3 How much space? The `sizeof` operator . . . 18

8. The char Type and ASCII 18

8.1 So... a character is just a number . . . 19

8.2 Non-printable characters (escape sequences) . . . 19

9. Floating-Point Types 20

10. Format Specifiers 20

11. Type Conversion and Casting 21

11.1 Automatic type conversion . . . 21

11.2 Type casting: explicit conversion . . . 21

12. Booleans 22

13. Be Mindful: Surprising Results 22

13.1 Unsigned subtraction . . . 22

13.2 Storing into narrower types and Booleans . . . 22

14. Self-Study Slides ("Study remaining slides yourself") 23

14.1 Examples: char constants . . . 23

14.2 Examples: integer and floating-point constants . . . 23

14.3 Integral promotion (ABC 3.11) . . . 24

14.4 Bitwise operators (ABC 7.1) . . . 24

14.5 Integers of specific sizes (C99) . . . 26

14.6 Strings . . . 26

15. Quick Reference Sheet 27

Toolchain commands . . . 27

Types at a glance (64-bit Linux/macOS) . . . 27

Gotchas checklist . . . 27

16. Practice Questions 28

Answers . . . 29

PART 1

Introduction to C

SPC1: context, toolchain, Hello World, and your first real program

1. C in Context

1.1 Lecture overview

The introduction lecture has three goals: place C in its historical and conceptual context, set up the tools you need, and get you writing and running real programs right away.

- **C in context, briefly** - where it came from and what kind of language it is
- **C resources** - the compiler, linker, and editor you will use
- **Getting started with examples** - Hello World, then a program that reads input and loops

1.2 Very brief history

The classic reference is *The C Programming Language* by **Kernighan & Ritchie**, universally called **"K&R"**. The original dialect described there is known as **K&R C**. C is **still evolving**: there are several standard versions.

Version	Year	Notes
K&R C	1978, 1988	Original book (1978) and its second edition (1988)
ANSI C	1989	First standard. Also called **C89**, or **C90** (ISO 9899:1990)
C99	1999	**The version this course mainly uses**
C11	2011	
C18	2018	
C23	2024	Most recent

> **KEY IDEA**
> We study **C99** mainly. That matters because several features you will meet in these slides are C99 additions: single-line `//` comments, the `_Bool` type, and the `<stdint.h>` fixed-size integer types.

"History for Larpers" (the Unix / C / Linux story)

Two slides reproduce a passage from *Computer Systems: A Programmer's Perspective* (the source cited on the slide). It is optional background (hence the joke title), but it explains **why C exists**. In summary:

- **1960s:** operating systems such as IBM's OS/360 and Honeywell's **Multics** were huge and complex. Multics dragged on for years without wide adoption. Bell Labs, an original partner, dropped out in **1969**.
- **1969:** Bell Labs researchers **Ken Thompson, Dennis Ritchie, Doug McIlroy and Joe Ossanna** began a *simpler* OS for a DEC PDP-7, written entirely in machine language. They kept good Multics ideas (the **hierarchical file system**, the **shell as a user-level process**) in a smaller package.
- **1970:** Brian Kernighan named it **"Unix"**, a pun on "Multics". The **kernel was rewritten in C in 1973**; Unix was announced publicly in 1974.

- **Spread:** Bell Labs gave schools the source code on generous terms. UC Berkeley added **virtual memory** and the **Internet protocols** (the **BSD** releases, Unix 4.xBSD). Bell Labs' own line became **System V**. Vendor systems like Sun's **Solaris** descended from these.
- **Mid-1980s:** vendors added incompatible features. **IEEE** sponsored a standardization effort, which Richard Stallman dubbed **POSIX**. POSIX standardizes the C interface for Unix system calls, shell programs and utilities, threads, and network programming.
- **1991:** Finnish graduate student **Linus Torvalds** announced a hobby Unix-like kernel ('won't be big and professional like gnu') on the `comp.os.minix` newsgroup, for 386/486 AT clones. He had already ported `bash` and `gcc` to it. Combined with the **GNU project**, Linux grew into a complete POSIX-compliant Unix system, now running on everything from handhelds to mainframes.

> **KEY IDEA**
> Takeaway: **C and Unix grew up together.** C was created to write an operating system, which is why it gives you direct control over memory and hardware.

1.3 Design principles

C was designed to be:

- **Simple**
- **Easy to compile**
- **Typed** (every value and variable has a type)
- Able to support **low-level memory access**
- **Ideal for embedded controllers, operating systems**, and similar systems code

...and yet it is **powerful** and **fast**.

> **INSTRUCTOR NOTE**
> The slide's speaker notes just say "[weakly]": C is *weakly typed*. It has types, but happily converts between them implicitly (you will see this in SPC2: automatic type conversion, integral promotion, assigning a double to an int, etc.).

1.4 Paradigm: purely procedural

C is a purely procedural language. There is no object orientation whatsoever. The central dogma:

- The main abstraction tool is the **procedure / function**, with a **caller** and a **callee**.
- A function **abstracts away "how things are done"**: the caller only needs to know *what* the function does.
- Programming therefore means **organizing processes as procedures** and **composing processes through procedure calls**.

From the slide notes: *procedural programming* is a kind of **imperative** programming in which the program is built from one or more procedures (also called subroutines or functions). When state changes are localized to procedures, or restricted to explicit arguments and return values, this is **structured programming**. Since the 1960s, structured and modular programming have been promoted to improve maintainability and quality of

imperative programs; object-oriented programming tries to extend that same approach.

Procedural programming in C

- C **adheres to this philosophy**.
- It generates **very efficient code**.
- It **exposes as many low-level details as you wish to see**.
- It gives you **full control over memory management - there is no garbage collector!**
- **The programmer is fully in charge.**

> **WATCH OUT**
> "Fully in charge" cuts both ways. Nothing frees your memory or checks your array bounds for you. Much of this course is about taking that responsibility seriously.

1.5 Resources and workflow

To write C you need only three things: **a C compiler, a linker, and a text editor.** The course uses the **GNU toolchain**:

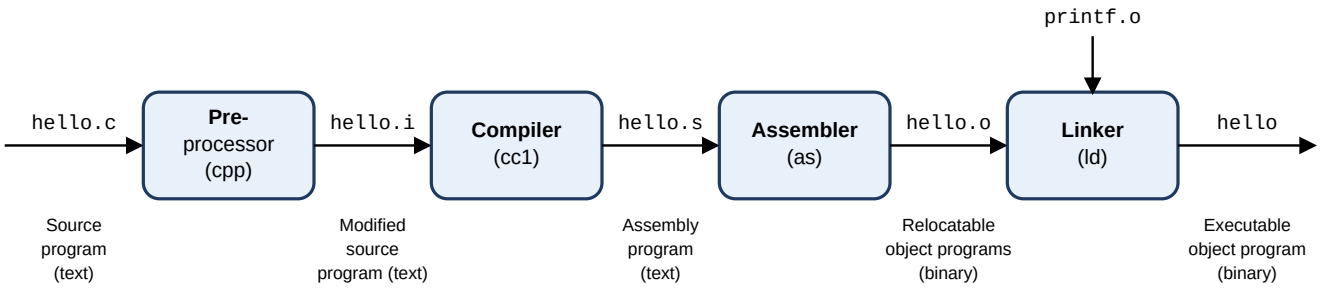
Role	Tool	Notes
Compiler	`gcc` (or `cc`)	The command you will run most
Linker	`ld`	You rarely call it directly; `gcc` **invokes it for you**
Editor	`vim`, `nano`, `emacs`, ...	Any plain text editor works

The workflow

- Use a **text editor** to edit **source files** (`.h` header files and `.c` source files).
- Use the **compiler** to generate **object files** (`.o`) from the `.c` files.
- Use the **linker** to link multiple `.o` files into an **executable**.

The compilation system (slide: "Students should look into this image")

The instructor explicitly wants you to study this figure (from the CS:APP textbook). It shows what really happens between `hello.c` and a runnable program:



Stage	Tool	Input	Output	Form
1. Preprocess	`cpp`	`hello.c`	`hello.i`	Modified source (text)
2. Compile	`cc1`	`hello.i`	`hello.s`	Assembly program (text)
3. Assemble	`as`	`hello.s`	`hello.o`	Relocatable object (binary)
4. Link	`ld`	`hello.o` + `printf.o`	`hello`	Executable object (binary)

The linker is where `printf` comes from: your `hello.o` only *calls* `printf`; the actual code lives in the library object `printf.o` (part of the C standard library, `libc`) and gets merged in at link time.

BEYOND THE SLIDES
You can stop `gcc` after each stage to inspect the intermediate files: `gcc -E hello.c` (preprocess only), `gcc -S hello.c` (stop at assembly, produces `hello.s`), `gcc -c hello.c` (stop at the object file `hello.o`). Plain `gcc hello.c` runs all four stages.

2. Hello, World! Dissected

```c
#include <stdio.h>

/* comments */
// single line comments

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

2.1 Comments

- The compiler **ignores everything between `/*` and `*/`**. Comments are meant to be **human readable**.
- Block comments can span **multiple lines** but **cannot be nested**.
- **Single-line comments (C99):** everything from `//` to the end of that line is ignored.

```c
/* my first program */

/********************
 * multiple lines
 ********************/

// Single line
```

WATCH OUT
Because block comments can't nest, `/* outer /* inner */ still outer? */` ends at the *first* `*/`. Don't use `/* */` to "comment out" code that already contains `/* */` comments.

2.2 The `main` function

- `main` is a **special function that defines the entry point** of the program. This is where the **operating system transfers control** once the program starts.
- `void` in the parameter list means **no arguments**.
- `int` before `main` means the function **returns an integer**.

Why return anything from main()?

When a process terminates it can **tell the OS how things went**; the return value of `main` is the way to report back. **Returning `0` means "everything went according to plan."** Any non-zero value signals some kind of problem.

> **BEYOND THE SLIDES**
> In a shell, check the last program's exit status with `echo $?`. Scripts and tools like `make` use this value to decide whether a step succeeded.

2.3 `printf`

- `printf` is a **C library function** that prints on the **standard output** of the process.
- It takes **at least one argument: a string**, written between double quotes, like `"This is a string."`.
- `\n` is the **newline** character.

2.4 Including header files

`#include <stdio.h>` "imports" a **header file** containing the *specification* (declarations) of functions that exist in a library which will be linked with your program. Here that includes `printf`. Without the proper header, the compiler doesn't know what `printf` looks like.

> **KEY IDEA**
> Rule from the summary slide: **include the proper header files for the library functions you use.**

2.5 Compile and execute

What is `gcc` doing, really? Three conceptual steps (the figure above breaks them down further):

- **Preprocesses** `hello.c`
- **Compiles** `hello.c` to `hello.o`
- **Links** `hello.o` with `libc`, and writes the executable file `a.out`

You can and often will **separate those steps**.

```
$ gcc hello.c
$ ./a.out
Hello, world!
```

Naming and running the executable

- `a.out` is the **default** name. Change it with `-o`.
- `./` means "the file in the **current directory**". Without it, the OS only searches the directories listed in your `PATH` (which normally doesn't include `.`).

```
$ gcc hello.c -o hello
$ ./hello
```

3. A Second Program: Sum from 1 to n

Purpose: read an integer `n` from standard input, compute the sum of all integers from 1 to `n`, and print the result on standard output. **New concepts:** standard input and output.

```
/*
  Compute the sum of the integers
  from 1 to n, for a given n
*/
#include <stdio.h>

int main(void) {
    int i, n, sum;
    sum = 0;
    printf("Enter n:\n");
    scanf("%d", &n);
    i = 1;
    while (i <= n) {
        sum = sum + i;
        i = i + 1;
    }
    printf("Sum from 1 to %d = %d\n", n, sum);
    return 0;
}
```

The slides walk through this one piece at a time. Here is each callout:

Piece	What it is	Explanation
`int main(void)`	Function header	`void` **tells the compiler the function takes no arguments**.
`int i, n, sum;`	Local variable declarations	Declares three `int` variables **inside** `main`, so they are **local variables**. Variables must be declared before use.
`scanf("%d", &n);`	Input	Reads an integer (`%d`) from the input and **stores it in** `n`.
`sum = 0; i = 1;`	Assignment expressions	Store a value into a variable. With a `;` they become statements.
`while (i <= n) { ... }`	Loop	Repeats the body **as long as the condition is true**; the braces group the statements in the body.
`sum += i;`	Compound assignment	Shorthand for `sum = sum + i;`
`i += 1; i++;`	Compound assignment / increment	Both are shorthand for `i = i + 1;`

Sample run (the slide runs it with input 100):

```
$ gcc sum.c
$ ./a.out
Enter n:
100
Sum from 1 to 100 = 5050
$
```

Tracing the loop (n = 4)

Check `i <= n`	`sum` after body	`i` after body
initially	0	1
1 <= 4 true	0 + 1 = 1	2
2 <= 4 true	1 + 2 = 3	3
3 <= 4 true	3 + 3 = 6	4
4 <= 4 true	6 + 4 = 10	5
5 <= 4 false	loop exits; prints 10	-

BEYOND THE SLIDES

In `scanf("%d", &n)`, the `&` means "the address of `n`". `scanf` needs to know *where* in memory to put the number, so you hand it the variable's address rather than its value. You will study this properly with pointers. Also, the closed-form check: 1 + ... + n = n(n+1)/2, and 100*101/2 = 5050.

3.1 Statements, functions and variables (the summary slide)

- A C program consists of **functions**.
- `main()` **is the entrance** of a program.
- A function consists of a **sequence of statements**.
- **Variables can be declared in a function** (like `main`): these are **local variables**.
- **Variables must be declared** before use.
- **Statements are terminated with** `;`.
- **Empty statements are allowed**, e.g. `;;;` is three empty statements.
- **Include proper header files** for library functions.

3.2 Self-study slides

The instructor marked the last two slides "Study the remaining slides yourself."

`main` taking arguments

- `main` can take **two** arguments: `argc` (the number of arguments) and `argv` (an array of the arguments).
- You will learn how to use them later.
- **Some systems support a third argument** (environment variables).

```
#include <stdio.h>

int main(int argc, char* argv[])
{
    printf("Hello world!\n");
    return 0;
}
```

BEYOND THE SLIDES

For `./prog a b`: `argc` is 3, and `argv[0]`="./prog", `argv[1]`="a", `argv[2]`="b". The first argument is always the program's own name. The common three-argument form is `int main(int argc, char *argv[], char *envp[])`.

if-else: compared with Python

Simple `if` statements are very similar to Python's. The differences: **parentheses around the condition**, `{` **and** `}` **to enclose multiple statements**, and **every statement ends with** `;`. The `else` branch works the same way. (More about if-else in the coming weeks.)

```
// C
if (a >= 0) {
    b = c + a;
    d = d + 5;
}
else {
    b = c - a;
    d = d + 10;
}
```

```
# Python
if a >= 0:
    b = c + a
    d = d + 5
else:
    b = c - a
    d = d + 10
```

WATCH OUT

In C, **indentation means nothing to the compiler** - braces do. Indent anyway, for humans.

PART 2

Expressions and Basic Data Types

SPC2: expressions, operators, integer/char/float types, conversions, and bit manipulation

4. Expressions in C

Expressions in C are similar to those in Java, C++ and Python. They are **inductively defined**:

- **Base cases:** constants, variables, and function calls (covered later).
- **Inductive step:** combine expressions using **parentheses and operators**.

Key facts:

- **All C expressions have a type**: constants have a type, variables have a type, function return values have a type, and **every sub-expression of a larger expression has a type**.
- **Adding a semicolon to an expression makes it a statement.**

```
5           // expression of type int
x + 5       // expression (type depends on x)
x + 5;      // statement: an expression followed by a semicolon
```

5. Basic Data Types, Constants and Variables

5.1 A few basic data types

Type	Meaning
`int`	An integer
`char`	A single byte that can store a character in ASCII. Also **an 8-bit integer**.
`float`	Floating point numbers

(More on basic data types later in the deck.)

5.2 Constants (of basic types)

Constants cannot be changed. They are written directly in the code (also called *literals*) and each has a type.

```
// char
'a', 'b', '\n'

// integer (the compiler stores them in binary)
200, -34
0x7fffFFFF    // hex
07112         // octal

// floating point numbers
3.1415, -0.34, 1.3E20
```

WATCH OUT

A leading **0** makes an integer constant **octal**. So `07112` is *not* seven thousand one hundred twelve, and `08` **is invalid** (8 is not an octal digit). Never pad integers with leading zeros in C.

5.3 Variables

- **All variables must be declared (and initialized) before use.**
- A declaration specifies the **type and name**; the **compiler allocates memory based on the type**.
- Valid names consist of **letters (case sensitive!), digits, and _**, but **cannot start with a digit**.
- **Multiple variables of the same type** can be declared together.
- Variables can be initialized **when declared** (a "variable definition") **or using separate assignments**.

```
char c;
int  i, j, k = 1;    // only k is initialized here
float f;
```

WATCH OUT

In `int i, j, k = 1;` only `k` gets the value 1. A local variable that is declared but never assigned holds an **indeterminate (garbage) value** - reading it is a bug. That is why the slide says to initialize before use. Also, `Sum` and `sum` are different variables.

6. Operators

6.1 The operator families

C has the conventional arithmetic, bitwise, and logical operators:

Family	Operators	Notes
Arithmetic	`+ - * / %`	`%` is remainder (integers only)
Bitwise	`& | ~ ^ << >>`	AND, OR, NOT (complement), XOR, shifts - see section 12
Logical	`&& || !`	AND, OR, NOT on truth values
Increment/decrement	`i++ ++i j-- --j`	Pre and post forms, as in Java/C++
Assignment	`= += *= ...`	Simple and compound; **more operators to come**

Pre- versus post-increment

Both add 1 to `i`. The difference is the **value of the whole expression**:

```
c = i++;   // c gets the OLD value of i, then i is incremented. (c will be i - 1 afterwards)
c = ++i;   // i is incremented first, then c gets the NEW value. (c will equal i)

int i = 5, c;
c = i++;     // c = 5, i = 6
c = ++i;     // i = 7, c = 7
```

BEYOND THE SLIDES
Integer division truncates toward zero: `7 / 2` is `3`, `-7 / 2` is `-3` (since C99), and `7 % 2` is `1`. For a real-valued result at least one operand must be floating point: `7 / 2.0` is `3.5`. Avoid modifying a variable twice in one expression (like `i = i++ + ++i`): the result is undefined.

6.2 Precedence and associativity

- **Precedence** determines **which operation is done first**.
- If operators have the **same precedence**, **associativity** decides (left-to-right or right-to-left).
- When in doubt, **use parentheses**.

```
 i + j * 10 - k / 20
(i + (j * 10)) - (k / 20)     // same thing, with the implicit grouping shown
```

The slide's table (listed from **most** to **least** binding):

Operators (highest precedence first)	Associativity
`()` `++` (postfix) `--` (postfix)	left to right
`+` (unary) `-` (unary) `++` (prefix) `--` (prefix)	right to left
`*` `/` `%`	left to right
`+` `-`	left to right
`=` `+=` `-=` `*=` `/=` etc.	right to left

Worked example of associativity: `a - b - c` is `(a - b) - c` (left to right), while `a = b = c = 0` is `a = (b = (c = 0))` (right to left).

BEYOND THE SLIDES
The slide's table is abbreviated. Full C ordering, highest to lowest: postfix `() [] . -> ++ --`; unary `! ~ + - ++ -- * & sizeof (cast)`; `* / %`; `+ -`; `<< >>`; `< <= > >=`; `== !=`; `&`; `^`; `|`; `&&`; `||`; `?:`; assignments; comma. Notable trap: `==` **binds tighter than** `&`, so write `(x & mask) == 0`, not `x & mask == 0`.

6.3 Assignment operators

- **Assignment:** `LHS = Expression`.
- The **LHS (Left Hand Side)** must be something that **can be written to** (e.g. a variable).
- LHS and Expression need **"compatible" types**.
- The value of Expression is stored into LHS **and also becomes the value of the whole assignment operation**.
- **Compound assignment operators** (`+=`, `*=`, ...): `var op= expr` means `var = var op expr`.

```
a = x + y;      b = c = d = 0;
i += 10;        // i = i + 10
j *= 5;         // j = j * 5
```

Assignments ARE NOT statements - they are expressions

`=` **is an operator**, and an assignment is an expression with a value. That means you can **chain** assignments and **use them inside larger expressions**. Each pair below is equivalent:

Compact form	Equivalent expanded form
`int a, b, c;` `a = (b = (c = 10));`	`c = 10; b = 10; a = 10;`
`int a, b, c;` `a = b = c = 10;`	(same as above - `=` is right to left, so the parentheses are optional)
`int a, b, c;` `a = (b = 2) + (c = 3);`	`b = 2; c = 3; a = b + c;` (so `a` is 5)

Slide note: in `a = 10;` the `a = 10` part is an **assignment expression**, and the `;` turns it into a **statement**.

> **WATCH OUT**
> Because assignment is an expression, `if (x = 5)` is *legal C*: it assigns 5 to `x` and the condition is then true. You almost certainly meant `if (x == 5)`. Compilers warn about it with `-Wall`.

7. Integer Data Types

C's integer types, in increasing size order (the second column shows allowed shorthand names):

Full name	Short name
`char`	`char`
`short int`	`short`
`int`	`int`
`long int`	`long`
`long long int`	`long long`

Each also has an `unsigned` version, such as `unsigned char`, `unsigned short`, etc.

How many bytes does each take? It depends on the CPU architecture and the compiler.

7.1 Sizes and ranges: x86_64 / arm64 (64-bit, "what we all have")

Size (bits)	Signed	Unsigned
8	`char`: -128 .. 127	`unsigned char`: 0 .. 255
16	`short`: -32768 .. 32767	`unsigned short`: 0 .. 65535
32	`int`: -2^{31} .. $2^{31} - 1$	`unsigned int`: 0 .. $2^{32} - 1$
64	`long`: -2^{63} .. $2^{63} - 1$	`unsigned long`: 0 .. $2^{64} - 1$
64	`long long`: -2^{63} .. $2^{63} - 1$	`unsigned long long`: 0 .. $2^{64} - 1$

7.2 Sizes and ranges: i386 (32-bit)

Size (bits)	Signed	Unsigned
8	`char`: -128 .. 127	`unsigned char`: 0 .. 255
16	`short`: -32768 .. 32767	`unsigned short`: 0 .. 65535
32	`int`: -2^{31} .. $2^{31} - 1$	`unsigned int`: 0 .. $2^{32} - 1$
32	`long`: -2^{31} .. $2^{31} - 1$	`unsigned long`: 0 .. $2^{32} - 1$
64	`long long`: -2^{63} .. $2^{63} - 1$	`unsigned long long`: 0 .. $2^{64} - 1$

KEY IDEA

The **only row that differs** between the two tables is `long`: 64 bits on a 64-bit machine, 32 bits on a 32-bit machine. That is exactly why the slide says size depends on architecture and compiler, and why section 13's fixed-size types exist.

Why those ranges? An *n*-bit **unsigned** integer holds 0 to $2^n - 1$. An *n*-bit **signed** (two's complement) integer holds -2^{n-1} to $2^{n-1} - 1$. For 8 bits: 0..255 and -128..127.

BEYOND THE SLIDES

Whether plain `char` is signed or unsigned is up to the implementation (signed on x86, unsigned on many ARM Linux targets), so use `signed char` / `unsigned char` when it matters. Also, on 64-bit **Windows**, `long` is 32 bits (the table above is the Linux/macOS convention).

7.3 How much space? The `sizeof` operator

- `sizeof` **gives the number of bytes** needed for a **type or a variable**: `sizeof(T)`.
- You will need this later to **dynamically allocate space**!

```
int i;
sizeof(int);   sizeof(i);    // 4 on the machines we use in this course
```

BEYOND THE SLIDES

`sizeof` is an **operator**, not a function, and it is evaluated at compile time. Its result type is `size_t`; print it with `%zu`: `printf("%zu\n", sizeof(int));`

8. The char Type and ASCII

- `char` **has 8 bits (a byte).**
- **ASCII code:** characters are mapped to an integer in **0..127**.
- Any ASCII character can be stored in a `char`.

Classes in ASCII

Decimal range	Class	Examples
0 .. 31 (and 127)	**Control** characters (non-printable)	0 = NUL, 7 = BEL, 8 = BS, 9 = TAB, 10 = LF (newline), 13 = CR, 27 = ESC; 127 = DEL
32	Space (first printable)	
48 .. 57	**Digits** `'0'` .. `'9'`	`'0'` = 48, `'9'` = 57
65 .. 90	**Upper case** letters	`'A'` = 65, `'H'` = 72, `'Z'` = 90
97 .. 122	**Lower case** letters	`'a'` = 97, `'z'` = 122

The slide shows the full ASCII chart (control characters 0-31 and 127, then printable 32-126). Want it in your own terminal? Run `man ascii`.

8.1 So... a character is just a number

The character `'H'` is none other than **72**. These two variables hold the **same value**:

```
char h1 = 'H', h2 = 72;    // h1 and h2 have the same value
```

Observe that `'0'` **through** `'9'` **are consecutive**, `'A'` **through** `'Z'` **are consecutive**, and `'a'` **through** `'z'` **are consecutive**. This enables arithmetic tricks:

```
char ch = '8';
int  x  = ch - '0';       // What is the value of x?   56 - 48 = 8
```

KEY IDEA

Answer: x is 8. `'8'` is 56 and `'0'` is 48, so subtracting `'0'` converts a digit *character* into the digit *value*. The same idea gives `'a' - 'A'` = 32, the gap between a lower-case letter and its upper-case form.

WATCH OUT

`'8'` (the character, value 56) and `8` (the integer) are different things. `'0'` is 48, not 0.

8.2 Non-printable characters (escape sequences)

Sometimes useful. Written as a **character constant (literal)**:

Literal	Meaning
`'\n'`	newline
`'\r'`	carriage return
`'\f'`	form feed
`'\t'`	tabulation
`'\b'`	backspace
`'\x7'`	audible bell (**x** indicates **hexadecimal**)
`'\07'`	audible bell (**0** indicates **octal**)

9. Floating-Point Types

A few floating-point types, again on x86_64:

Size (bits)	Size (bytes)	Name & range
32	4	`float`: 1.17×10^{-38} to $3.40 \times 10^{+38}$
64	8	`double`: 2.22×10^{-308} to $1.79 \times 10^{+308}$
80/128	16	`long double`: 3.65×10^{-4951} to $1.18 \times 10^{+4932}$

`long double` is stored in 16 bytes (128 bits) but on x86 only **80 bits** carry data (extended precision).

> **BEYOND THE SLIDES**
> These ranges are for the smallest **positive normalized** magnitude up to the largest. Floats only approximate real numbers: `0.1 + 0.2` is not exactly `0.3`. Use `double` by default; `float` is rarely worth the lost precision on a desktop CPU.

10. Format Specifiers

A format specifier **tells the compiler/library the type of the data** to be printed (with `printf`) or scanned (with `scanf`). Here are a few; there are plenty more to learn later.

Specifier	Description
`%d`	`int`
`%f`	`float`
`%c`	`char`
`%ld`	`long`
`%lf`	`double`
`%o`	Octal representation
`%x`	Hexadecimal representation

```
int  n = 255;   char ch = 'A';   double d = 3.5;
printf("%d %x %o\n", n, n, n);   // 255 ff 377
printf("%c %d\n", ch, ch);       // A 65     (same char, shown two ways)
printf("%lf\n", d);              // 3.500000
```

WATCH OUT

A specifier that **doesn't match the argument's type** gives garbage or undefined behavior (e.g. `%d` with a `double`). Compile with `-Wall` to get warnings.

BEYOND THE SLIDES

With `scanf`, `%f` reads a `float` but you **must** use `%lf` to read a `double`. With `printf`, `%f` works for `double` too (floats are promoted to double when passed).

11. Type Conversion and Casting

11.1 Automatic type conversion

When an operator has operands of **different types**, the compiler **automatically converts them to a common type**. In general the **lower-rank operand is converted to the type of the higher-rank one**, where:

`char < short < int < long < long long < float < double < long double`

- E.g. in `1 + 2.5`, the `1` is converted to `double` before the addition, so the result is `3.5`.
- Conversion also happens **across assignments**: the right-hand side may be **widened** to the type of the LHS, e.g. `double d = 1;`
- ...or **narrowed (possibly with information loss)**, e.g. `int i = 2.5;` (the fractional part is dropped, so `i` becomes 2).

WATCH OUT

Widening (small to big) is generally safe. **Narrowing** (big to small) can silently lose data.

11.2 Type casting: explicit conversion

Useful to convert an operand to another type **before** doing arithmetic. Syntax: `(<Type>)<expression>`.

Example - is the result an integer or a double?

```
int x = 10;
int y = 3;
double z = x / y;            // z = 3.0   (integer division happens first!)

int x = 10;
int y = 3;
double z = (double)x / y;    // z = 3.333...   (x is converted first; y is promoted to match)
```

Why: in the first version both operands are `int`, so `x / y` is **integer division** (3); only *then* is that 3 converted to `3.0` for the assignment. In the second, the cast applies to `x` alone (casts bind tighter than `/`), so the division is performed in `double`.

WATCH OUT
The slide's comment: `z = (double)(x / y)` **doesn't work** - meaning it *does not give the 3.333 you want*. The cast wraps the whole division, which is *already done in integers* (3), so you just get `3.0`. Cast an **operand**, not the **result**.

12. Booleans

- **K&R C and C89/C90 do not have a Boolean data type.**
- `0` **means FALSE; anything else means TRUE.**
- It is common to use `int` or `char` for Boolean values, and to define convenience **macros**.
- **C99 introduced** `_Bool`: a variable of `_Bool` type is **either 0 or 1**.

```
#define BOOL   char
#define TRUE   1
#define FALSE  0
```

BEYOND THE SLIDES
In C99 you can `#include <stdbool.h>` to get the friendlier names `bool`, `true` and `false` (which expand to `_Bool`, 1 and 0). Comparison and logical operators (`<`, `==`, `&&`, `!`, ...) always produce an `int` that is 0 or 1.

13. Be Mindful: Surprising Results

Sometimes the results may not be what you expected. Always ask: **What is the size (in bits) of each operand? Are your operands signed or unsigned?**

13.1 Unsigned subtraction

```
unsigned int x = 3;
unsigned int y = 7;
unsigned int z = x - y;
```

`z` holds the **binary representation of -4**, but **reading it as an** `unsigned int` **yields a very different value**: it wraps around to 2^{32} - 4 = **4,294,967,292** (`0xFFFFFFFC`). Unsigned arithmetic never goes negative - it wraps modulo 2^n.

13.2 Storing into narrower types and Booleans

```
_Bool b1;
char  b2, b3;
int   i = 256;        // 0x100

b1 = i;
b2 = i;
b3 = i != 0;
```

Variable	Value	Why
`b1`	1	Converting to `_Bool` gives 1 for **any non-zero** value, and `i` is not 0.
`b2`	0	A `char` keeps only the **lowest 8 bits** of `i`. `256` = `0x100`, and its low 8 bits are all 0.
`b3`	1	`i != 0` is a comparison yielding 1 (true), and `i` is not 0.

"Do you want `b2` or `b3`?" To record *whether* a value is non-zero, use the comparison (`b3`), not a plain assignment into a narrow type (`b2`), which truncates.

14. Self-Study Slides ("Study remaining slides yourself")

14.1 Examples: char constants

Single quotation marks, exactly **one character** (or one escape sequence):

```
'h'
'\n'
'\007'     // octal
'\xAA'     // hex. 170 = 0xAA
'\''       // single quotation mark
'\\'       // back slash
'"'        // no need to escape double quotation mark here
```

14.2 Examples: integer and floating-point constants

```
200
-300
0x7fffFFFFu    // hex. unsigned int. case insensitive
0123456        // octal. starting with 0!
0x12345678L    // hex. long int
123UL          // unsigned long
123LL          // long long
12345678901234567890ull    // unsigned long long
3.14f          // float literals
3.14L          // long double literals
```

Suffix / prefix	Meaning
`0x` / `0X`	hexadecimal (digits 0-9, a-f, case-insensitive)
leading `0`	octal
`u` / `U`	unsigned
`l` / `L`	long (on a float literal, `L` means `long double`)
`ll` / `LL`	long long
`f` / `F`	float (otherwise a decimal literal like `3.14` is a `double`)

Without a suffix, a floating-point constant like `3.14` is a `double`. Suffixes can be combined (`UL`, `ull`, ...).

14.3 Integral promotion (ABC 3.11)

Integer types smaller than `int` (such as `char` or `short`) are **promoted to** `int` **(or** `unsigned int`**)** when an operation is performed on them.

```
// If there is no integral promotion,
// c1 * c2 would not have 600 as result
char r, c1, c2, c3;
c1 = 100;
c2 = 6;
c3 = 8;
r = c1 * c2 / c3;
```

`c1 * c2` is computed as `int`: 100 * 6 = **600** (which would not fit in a `char`, max 127). Then 600 / 8 = **75** (still as `int`), and 75 fits when stored into `r`. Without promotion, 600 would have overflowed a `char` before the division.

14.4 Bitwise operators (ABC 7.1)

Op.	Description	Typical use
`&`	bitwise AND	**Set bits to 0. Mask out** bits
`|`	bitwise OR	**Set bits to 1**
`^`	bitwise XOR	**Flip some bits** (using masks)
`~`	1's complement	**Flip all bits**
`<<`	shift left	Move bits to the left
`>>`	shift right	Move bits to the right (**pay attention to the sign**)

All of these can be suffixed with `=`: for instance, `a &= b;` **is the same as** `a = a & b;`.

WATCH OUT

Don't confuse `&` **/** `|` **with** `&&` **/** `||`**.** `&&` and `||` are logical (whole-value true/false); `&` and `|` work bit by bit. `1 & 2` is 0 but `1 && 2` is 1.

BEYOND THE SLIDES

Shift right on a **signed negative** value is implementation-defined: most compilers copy the sign bit (*arithmetic* shift), while unsigned values get zeros (*logical* shift). That's what "pay attention to the sign" refers to. Prefer unsigned types for bit manipulation.

Bitwise examples (8-bit values)

```
  11010011          11010011          11010011
& 10001100        | 10001100        ^ 10001100
----------        ----------        ----------
  10000000          11011111          01011111
    AND               OR                XOR

 11010011 << 3  =  10011000     (bits move left; 3 zeros enter on the right,
                                 the 3 leftmost bits fall off the 8-bit value)
~11010011       =  00101100     (every bit flipped)
 01010011 >> 3  =  00001010     (bits move right; 3 zeros enter on the left)
```

Reading the AND: a result bit is 1 only where **both** inputs have 1. OR: where **either** has 1. XOR: where the inputs **differ**.

Example: getting (testing) a bit

Suppose **bit 2** (the third bit from the right, counting from bit 0) of `lights` says whether a light is on (1) or off (0). **Hexadecimal is shorter and easier to read**: each hex digit is exactly 4 bits, so `0x27` = `0010 0111`.

```
char lights = 0x27;         // lights: 00100111
char mask = 0x1;            // mask:   00000001
mask <<= 2;                 // mask:   00000100   (a 1 in position 2 only)
if (lights & mask)          // lights & mask: 00000100  -> non-zero -> true
    printf("room 2 is on");
else
    printf("room 2 is off");
```

The mask has a single 1 in the position we care about. ANDing keeps only that bit of `lights`; the result is non-zero exactly when that bit was 1. Here it prints **"room 2 is on"**.

> **WATCH OUT**
> On the slide, the string in `printf` uses curly "smart quotes" (copied from PowerPoint). **Real C code needs straight quotes `"..."`.** Copy-pasting from slides is a classic source of baffling compile errors.

Example: setting a bit

```
char lights = 0x0;          // lights: 00000000
char mask = 0x1;            // mask:   00000001
mask <<= 2;                 // mask:   00000100
lights |= mask;             // lights: 00000100   (bit 2 is now 1)
```

OR with a mask sets the chosen bit to 1 and leaves all other bits unchanged.

> **BEYOND THE SLIDES**
> The two remaining standard operations: **clear** bit 2 with `lights &= ~mask;` and **toggle** it with `lights ^= mask;`. A compact way to build the mask is `(1 << 2)`.

Goal	Operation	Idea
Test a bit	`if (x & mask)`	AND keeps just that bit
Set a bit to 1	`x |= mask;`	OR with 1 forces 1
Clear a bit to 0	`x &= ~mask;`	AND with 0 forces 0 (`~mask` is all 1s except that bit)
Toggle a bit	`x ^= mask;`	XOR with 1 flips

14.5 Integers of specific sizes (C99)

Because `short`/`int`/`long` sizes vary by platform, C99 provides fixed-size types in `<stdint.h>`:

```
#include <stdint.h>
int8_t         // signed 8 bits integers
int16_t
int32_t
int64_t
uint8_t        // unsigned 8 bits integers
uint16_t
uint32_t
uint64_t
// Many projects have their own *standard* types
```

`int<N>_t` is a signed integer of exactly *N* bits; `uint<N>_t` is unsigned. Use them when exact width matters (hardware registers, file formats, network protocols, bit manipulation). The last comment notes that many projects define their own standard type names (often typedefs on top of these).

14.6 Strings

- **A string is *not* a basic type.**
- A string is an **array of characters** (to be discussed more later; the instructor's note on this slide is just "Arrays").

```
"Hello world!\n"
"Hello " "world!\n"      // Useful for long strings (adjacent literals are joined)
"It's \"Mickey\"\n"      // Escape double quotations
"A bell \007\n"          // Can use ASCII code
```

BEYOND THE SLIDES

Note the quote rules: **double quotes** make a string (`"A"`), **single quotes** make one character (`'A'`). They are different types. In a string, a single quote needs no escaping but a double quote does; in a `char` constant it's the reverse. A string in memory also ends with a hidden zero byte `'\0'` (the *terminator*), so `"Hi"` occupies 3 bytes.

15. Quick Reference Sheet

Toolchain commands

```
gcc hello.c            # compile + link, output a.out
gcc hello.c -o hello   # choose the output name
./hello                # run it from the current directory (./ is required)
echo $?                # show the exit status (value returned by main)
man ascii              # print the ASCII table in your terminal
```

Types at a glance (64-bit Linux/macOS)

Type	Bytes	Typical use / specifier
`char`	1	characters, small ints, `%c`
`short`	2	`%hd`
`int`	4	default integer, `%d`
`long`	8	`%ld`
`long long`	8	`%lld`
`float`	4	`%f`
`double`	8	default real number, `%lf` (scanf) / `%f` (printf)
`long double`	16	extended precision
`_Bool`	1	0 or 1 (C99)

Gotchas checklist

- `i++` vs `++i`: the *value* of the expression differs.
- Integer division truncates: `10 / 3` is `3`. Cast an operand: `(double)x / y`.
- Leading `0` = octal. `08` is invalid.
- `=` assigns, `==` compares.
- `&`/`|` are bitwise, `&&`/`||` are logical.
- Unsigned subtraction wraps around; narrowing assignment keeps only low bits.
- Block comments `/* */` don't nest.
- Variables must be declared (and given a value) before use. Statements end in `;`.
- `'A'` (char) is not `"A"` (string).
- Straight quotes only in real code.

16. Practice Questions

1. What does `return 0;` at the end of `main` communicate, and to whom?

2. What is the difference between `gcc hello.c` and `gcc hello.c -o hello`? How do you run each?

3. Name the four stages from `hello.c` to an executable, with each stage's tool and output.

4. After `int i = 5, c; c = i++;` what are `c` and `i`? And after `c = ++i;` starting again from `i = 5`?

5. Evaluate: `7 / 2`, `7 % 2`, `7 / 2.0`, `(double)7 / 2`.

6. After `int a, b, c; a = (b = 2) + (c = 3);` what are a, b, c?

7. What is `'z' - 'a'`? What is `'7' - '0'`?

8. `unsigned int x = 3, y = 7; unsigned int z = x - y;` What is z (on a 32-bit `int`)?

9. `char c1 = 100, c2 = 6, c3 = 8; char r = c1 * c2 / c3;` What is r, and why does it work?

10. `int i = 256; char b2 = i; char b3 = (i != 0);` What are b2 and b3?

11. Compute `0xD3 & 0x8C`, `0xD3 | 0x8C`, `0xD3 ^ 0x8C`.

12. Write code to (a) test, (b) set, (c) clear bit 5 of an `unsigned char b`.

13. Why does `double z = x / y;` with `int x = 10, y = 3;` give 3.0? How do you get 3.333...?

14. Is `08` a valid C integer constant? What is `010`?

15. Why is `sizeof(long)` not the same everywhere? Which fixed-size type would you use for an exact 32-bit unsigned value?

Answers

1. To the operating system: the program finished successfully.

2. The first writes `a.out`, run with `./a.out`; the second writes `hello`, run with `./hello`. `./` is needed because the current directory is normally not in `PATH`.

3. Preprocessor `cpp` -> `hello.i`; compiler `cc1` -> `hello.s`; assembler `as` -> `hello.o`; linker `ld` -> `hello` (links in `printf.o` from libc).

4. First: `c = 5, i = 6`. Second: `i = 6, c = 6`.

5. `3`, `1`, `3.5`, `3.5`.

6. `a = 5, b = 2, c = 3`.

7. 25 (122 - 97) and 7 (55 - 48).

8. 4294967292 (`0xFFFFFFFC`), since unsigned arithmetic wraps modulo 2^{32}.

9. 75. The operands are promoted to `int`, so 100*6 = 600 doesn't overflow; 600/8 = 75 fits in a `char`.

10. `b2 = 0` (low 8 bits of 0x100), `b3 = 1`.

11. `0x80`, `0xDF`, `0x5F` (the 11010011 / 10001100 example).

12. (a) `b & (1 << 5)`; (b) `b |= (1 << 5);`; (c) `b &= ~(1 << 5);`

13. Integer division happens first. Cast an operand: `(double)x / y`.

14. `08` is invalid (8 isn't an octal digit). `010` is octal, i.e. decimal 8.

15. Sizes depend on architecture and compiler (`long` is 64-bit on x86_64/arm64 Linux, 32-bit on i386). Use `uint32_t` from `<stdint.h>`.